CS683: Advanced Computer Architecture

Programming Assignment 2 - Report & Summary

Siddhant Kaul Pratham Srivastava Dhruv Singla Harshit Somani

Autumn 2026

Contents

1 Task 1A: Understanding IP-Stride Prefetcher **2**

2 Task 1B: Implementing and Evaluating the Stride Pattern Prefetcher (SPP) **2**

2.1 Problem and Approaches 2

1 Task 1A: Understanding IP-Stride Prefetcher

The `ip_stride.l1d_pref` module defines an `IP_TRACKER` class that represents a single row in the IP-stride table. It defines `l1d_prefetcher_initialize`, `l1d_prefetcher_operate`, `l1d_prefetcher_cache_fill`, `l1d_prefetcher_final_stats` under the `CACHE` namespace, which the ChampsSim simulator will ostensibly call at the appropriate level of cache hierarchy, with the correct arguments.
As explained by Prof. Biswa in class, the IP-stride prefetcher tracks the access pattern's 'stride' that is observed emanating from a particular instruction address. If there is a strong indication of the stride being repeated, indicating a linear access pattern, the microarchitecture prefetches the next element following the pattern (latest address + observed stride).
`IP_TRACKER trackers[IP_TRACKER_COUNT];` is defined, representing `IP_TRACKER_COUNT` rows of the table, each storing the tracked IP, previously accessed address, previous stride, and an index for LRU insertion/eviction of rows.
`l1d_prefetcher_operate` is the main callback that is executed every time a load reaches L1D, and contains the core logic of prefetching and table maintenance. When an address-IP pair arrives, it first checks if the IP is already being tracked (by looping through the table). If it is not, the LRU IP is evicted, the new IP is set to be the MRU, and the other indices' LRU indices are updated.
Next, the stride for the new IP is calculated. [1] If it is equal to the last stride observed on that IP, prefetching ensues. If not, the standard address-last address, LRU and stride updations take place.
For prefetching, `prefetch_line` is called for the next address (as per the current address stride), only if the next address is in the same 4kB page. Depending on MSHR occupancy, prefetching is done to L1 (if not full) or L2 (if full).

2 Task 1B: Implementing and Evaluating the Stride Pattern Prefetcher (SPP)

2.1 Problem and Approaches

The task is broadly to implement and evaluate a prefetcher capable of recognising non-uniform stride patterns or sequences, such as $(+1, +2, +3, +1, +2)^n$ or the like. For this particular example the sequence length is 5; in general, it could be anything. Additionally, the problem statement requires us to implement accuracy-based throttling of prefetch requests. The initial challenge is to implement a prefetcher that can identify a uniform pattern as well as a repeating sequence of a length that is not known a priori.

My first line of thinking was to maintain (per-IP) a rolling window of the last `2*MAX_PERIOD` observed strides, where `MAX_PERIOD` is the maximum length of the repeating sequence (the maximum possible period) the prefetcher will detect[2]. Then, from that window, both the period and the sequence itself can be determined. This can be done by checking, for each `i = 0...2*MAX_PERIOD`, whether `strides[i]` = `strides[i+l] = strides[i+2*l] = ..`, within the limits, where `strides` is the aforementioned rolling window of observed strides. If this is true for some `l`, then `l` is our stride, and we prefetch accordingly. This will, in fact, accurately detect whenever there exists a repeating sequence of length $\leq$ `MAX_PERIOD`. There are two issues with this, which are the reason I did not implement this architecture:

- This requires $\binom{2*\text{MAX_PERIOD}}{2}$ equality checks, followed by some Boolean logic, to detect the period for each rolling window. There are probably ways to pipeline this logic so you don't actually require that many comparator circuits[3] in the hardware, but I did think the actual combinational circuit to be implemented would be expensive, hardware-wise, and scale poorly with `MAX_PERIOD`.
- Storing `2*MAX_PERIOD` seemed to be more storage than strictly required to detect up to `MAX_PERIOD`.

[1] If it is a newly inserted IP, the function `return`s before reaching this part.
[2] We cannot detect arbitrarily long periods, because that necessarily requires arbitrarily large storage.
[3] Just a one-byte XOR, since we only have to check for equality.

For these reasons, I moved on from this initial approach. The approach implemented is described below, excluding accuracy throttling.

Maintain, as in IP-stride, a table of IPs and associated stride data. Maintain a rolling window of the last (importantly) `MAX_PERIOD` observed strides. Maintain saturating counters[4] for different 'candidate' periods, upto `MAX_PERIOD`, including period = 1 (uniform). When a new stride arrives, check for equality with all the stored strides. For each of the 'candidate' periods (1 to `MAX_PERIOD`), if `strides[current - period] = strides[current]`, then increment the saturating `matches[period]`. If it is not equal, reset `matches[period]` to zero. Finally, update the rolling window, throwing out the oldest value and bringing in the newest.

Firstly, this *works* because it will keep repeatedly penalise incorrect periods (which can be done by resetting to zero, or decrementing by a quantity ≥ 1), and invariably saturates the counter associated with the correct period, if the sequence repeats perfectly. Secondly, this reduces the size of the window that has to be stored by 50%. Thirdly, the number of equality checks needed every time a new stride comes goes down to `MAX_PERIOD`.

[4]Indicating confidence values the prefetcher associates with different possible period based on its observations